

CMCS Project 27

Vincenzo Gargano, Lorenzo Pace

January 2, 2025

Contents

1	Problem Statement	2
2	GMRES	3
2.1	Background: Krylov Spaces and Arnoldi	3
2.2	Simplifying the minimum problem	3
2.3	Lanczos	4
2.4	Lanczos + QR	5
2.4.1	Extending the QR decomposition	5
2.5	Solving the minimum problem, given QR	7
2.6	Computational Complexity	7
2.6.1	Time Complexity of Lanczos+QR	7
2.6.2	Space Complexity of Lanczos+QR	7
2.6.3	Time complexity of GMRES	8
2.6.4	Space complexity of GMRES	8
2.7	Stability	8
2.7.1	Background: Backward Stability	8
2.7.2	Backward Stability of QR factorization	8
3	GMRES with preconditioner	10
3.1	Implementation details	10
3.1.1	Complexity and stability	10
4	Additional implementations	11
4.1	Restarted versions of the algorithms	11
5	Generating instances of the problem	12
5.1	Min-Cost Flow	12
5.2	Problem and constraints in matrix form	12
5.3	KKT System	13
5.4	Netgen dataset, and generating D	13
6	Experimental Results	14
6.1	Comparison in performance for different D	14
6.2	Comparison between the preconditioned and non-preconditioned systems	14
6.3	Comparison between GMRES and Restarted GMRES	16

1 Problem Statement

(P) is a sparse linear system of the form:

$$\begin{bmatrix} D & E^T \\ E & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$$

where $D \in \mathbb{R}^{m \times m}$ is a diagonal positive definite matrix (i.e., $D = \text{diag}(D) > 0$) and $E \in \mathbb{R}^{(n-1) \times m}$ is obtained by removing the last row from the node-arc incidence matrix of a given connected directed graph. These problems arise as the KKT system of the convex quadratic separable Min-Cost Flow Problem, hence you can look, e.g., here for ways to generate meaningful instances of the problem.

- (A1) is GMRES, and you must solve the internal problems $\min \|H_n y - \|b\|e_1\|$ by updating the QR factorization of H_n at each step: given the QR factorization of H_{n-1} computed at the previous step, apply one more orthogonal transformation to compute that of H_n .
- (A2) is the same GMRES, but using the so-called *Schur complement preconditioner*

$$P = \begin{bmatrix} D & 0 \\ E & S \end{bmatrix}$$

where S is either $S = ED^{-1}E^T$ or a sparse approximation of it (to obtain it, for instance, replace the smallest off-diagonal entries of S with zeros).

No off-the-shelf solvers allowed.

2 GMRES

GMRES is an iterative method for computing an approximation of the solution to the linear system $Ax = y$. It accomplishes such approximation by computing:

$$\min_{x \in K_n(A, y)} \|Ax - y\|$$

i.e. by finding the minimum x within the *Krylov Space* $K_n(A, y)$ such that the norm of the residual $Ax - y$ is minimized.

2.1 Background: Krylov Spaces and Arnoldi

Given a non-necessarily-symmetric matrix (although in our case it is) $A \in \mathbb{R}^{m \times m}$, a vector $y \in \mathbb{R}^m$, and a natural number $n \leq m$:

$$K_n(A, y) = \text{span}(y, Ay, A^2y, \dots, A^{n-1}y).$$

We can obtain a good basis for this vector space by using the Arnoldi Algorithm:

$$\text{Arnoldi}(A, y, n) =: Q_n, \underline{H}_n$$

Where:

- $Q_n \in \mathbb{R}^{n \times m}$ is the matrix $(q_1 \mid \dots \mid q_n)$ of vectors in the basis;
- $\underline{H}_n \in \mathbb{R}^{n+1 \times n}$ is a matrix such that:

1.

$$\underline{H}_n = \begin{bmatrix} H_n \\ \dots \end{bmatrix}$$

where $H_n \in \mathbb{R}^{n \times n}$ is s.t. $H_n = Q_n^T A Q_n$, and can therefore be seen as A in the Krylov subspace.

2.

$$A Q_n = Q_{n+1} \underline{H}_n \tag{1}$$

Remark 1. The first vector of the basis generated by $\text{Arnoldi}(A, y, n)$ is $\frac{y}{\|y\|}$.

2.2 Simplifying the minimum problem

We can simplify the minimum problem to:

$$\min_{z \in \mathbb{R}^{n+1}} \|\underline{H}_n z - e_1 \|y\|\|$$

Proof.

$$\begin{aligned} \min_{x \in K_n(A, y)} \|Ax - y\| &= \min_{z \in \mathbb{R}^{n+1}} \|A Q_n z - y\| && (q_i \text{ s form basis of } K_n(A, y)) \\ &= \min_{z \in \mathbb{R}^{n+1}} \|Q_{n+1} \underline{H}_n z - y\| && (\text{by (1)}) \\ &= \min_{z \in \mathbb{R}^{n+1}} \|Q_{n+1}^T (Q_{n+1} \underline{H}_n z - y)\| && (Q_{n+1}^T \text{ orthogonal}) \\ &= \min_{z \in \mathbb{R}^{n+1}} \|\underline{H}_n z - Q_{n+1}^T y\| && (Q_{n+1} \text{ orthogonal}) \\ &= \min_{z \in \mathbb{R}^{n+1}} \|\underline{H}_n z - Q_{n+1}^T q_1 \|y\|\| && (\text{by Remark 1}) \\ &= \min_{z \in \mathbb{R}^{n+1}} \|\underline{H}_n z - e_1 \|y\|\| && (Q_{n+1}^T \text{ orthogonal}) \end{aligned}$$

□

2.3 Lanczos

Since the input matrix A is symmetric, we can use the Lanczos Algorithm to iteratively construct the orthonormal basis for the Krylov space. In particular, the H_n matrix resulting from such algorithm will be symmetric and tridiagonal:

$$H_n = \begin{bmatrix} \alpha_1 & \beta_1 & & & & \\ \beta_1 & \alpha_2 & \beta_2 & & & \\ & \beta_2 & \alpha_3 & \ddots & & \\ & & \ddots & \ddots & \beta_{m-2} & \\ & & & \beta_{m-2} & \alpha_{m-1} & \beta_{m-1} \\ & & & & \beta_{m-1} & \alpha_m \end{bmatrix}$$

This allows us to perform the computations without storing the whole matrix, but only the diagonal and subdiagonal. The Julia implementation of the algorithm is as follows: ¹

```

1 function Lanczos(A::SparseMatrixCSC, y::SparseVector, n::Int)
2
3     alpha = zeros(1,n)      # diagonal of H
4     beta = zeros(1,n)      # subdiagonal of H (= superdiagonal of H because symmetric)
5
6     L = zeros(size(A)[1], n) # Lanczos Basis
7
8     # base case
9
10    q = y / norm(y)
11    L[:, 1] = q
12    w = A * q
13    alpha[1] = w' * q
14    w = w - alpha[1] * q
15    beta[1] = norm(w)
16    # inductive case
17
18    for j in 2:n
19        if j % 500 == 0
20            println(j, "-th Lanczos iteration")
21        end
22        if beta[j-1] < 1e-13
23            println("Breakdown");
24            return
25        end
26        q = w / beta[j-1]
27        L[:, j] = q
28
29        w = A * q
30        alpha[j] = w' * q
31        w = w - alpha[j] * q - beta[j-1] * L[:, j-1]
32        beta[j] = norm(w)
33
34    end
35
36    return (L, alpha, beta)
37
38 end

```

¹ **TODO:**
Write this as
pseudocode

2.4 Lanczos + QR

A trivial way to implement GMRES would be:

- Use Lanczos to obtain a basis for the Krylov space
- Compute the QR factorization of $\underline{H}_n$
- Use the factorization to easily solve the linear system arising from the minimum problem.

However, the requested version should compute the QR factorization during the Lanczos iteration.

2.4.1 Extending the QR decomposition

Assume we have $\underline{H}_{n-1} = Q_{old}R_{old}$. The matrix $\underline{H}_n$ will be computed using a Lanczos iteration, obtaining:

$$\underline{H}_n = \left[\begin{array}{c|c} & \begin{matrix} 0 \\ \vdots \\ 0 \\ \beta_{n-1} \\ \alpha_n \end{matrix} \\ \hline \begin{matrix} 0 & \dots & 0 \end{matrix} & \beta_n \end{array} \right]$$

We want to find two matrices Q, R such that:

- $Q \in \mathbb{R}^{(n+1) \times (n+1)}$ is orthogonal
- $R \in \mathbb{R}^{(n+1) \times n}$ is upper triangular
- $\underline{H}_n = QR$, i.e. $Q^T \underline{H}_n = R$

First off, to obtain a Q matrix of the requested shape that is orthogonal, we could consider:

$$Q'_{old} = \left[\begin{array}{c|c} & \begin{matrix} 0 \\ \vdots \\ 0 \end{matrix} \\ \hline \begin{matrix} 0 & \dots & 0 \end{matrix} & 1 \end{array} \right]$$

This takes us close to the solution, but not quite there; in fact the product $Q'^T_{old} \underline{H}_n$ is *almost* triangular:

$$Q'^T_{old} \underline{H}_n = \left[\begin{array}{c|c} R_{old} & Q'^T_{old} \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \beta_{n-1} \\ \alpha_n \end{bmatrix} \\ \hline \begin{matrix} 0 & \dots & 0 \end{matrix} & \beta_n \end{array} \right]$$

Recall that R_{old} is an $n \times (n-1)$ upper triangular matrix, hence it has a row of zeros at the bottom. To make this matrix triangular, we should have a zero in place of the β_n . We can get rid of such zero by performing a Householder reflection on the bottom two elements of the final column. Let these be:

$$x = \begin{bmatrix} a \\ \beta_n \end{bmatrix} \quad \mapsto \quad y = \begin{bmatrix} \|x\| \\ 0 \end{bmatrix}$$

As seen in class, the transformation we need is a reflection w.r.t $v = x - y$:

$$H_{refl} = I - \frac{2vv^T}{\|v\|^2}$$

In order for the reflection to only act on the two bottom elements of the final row, we extend it as follows, obtaining another orthogonal transformation:

$$O = \left[\begin{array}{c|cc} I & 0 & 0 \\ \hline 0 \dots 0 & 0 & 0 \\ 0 \dots 0 & H_{refl} \end{array} \right]$$

The matrix we obtain as $OQ'_{old} \underline{H}_n$ is triangular.

Constructing the R Matrix

- Get old R ;
- Compute:

$$Q'_{old}[0, \dots, 0, \beta_{n-1}, \alpha_n]^T = ([0, \dots, 0, \beta_{n-1}, \alpha_n]Q'_{old})^T$$

Let $q_1 \dots q_n$ be the columns of Q'_{old} . ⁽²⁾ The result is:

$$\sum_i \beta_{n-1} q_{i,n-1} + \alpha_n q_{i,n}$$

This can be rewritten, letting $r_1 \dots r_n$ be the rows of Q'_{old} , as:

$$\beta_{n-1} r_{n-1} + \alpha_n r_n$$

- Perform the O transformation to obtain R .

Constructing the Q matrix

- Get old Q ;
- Construct Q'_{old} by adding the extra row and column;
- Compute $Q = (OQ'_{old})^T$.

Altenatively we can calculate the new Q as follows:

$$Q = O^T Q'_{old} = OQ_{old}$$

By the symmetry of O , which is guaranteed by the symmetry of H_{refl} . Computationally, we only need to update the last two columns of Q , so we need not construct O explicitly. The following Julia snippet shows such computation ³

```

1  Q[j+1, j+1] = 1.0
2
3  newcols = zeros(j+1, 2)
4
5  for i in 1:j+1
6      # Compute Q_old * 0 without constructing 0
7      # Can compute Q_old * 0 instead of Q_old * 0' because 0 is sym.
8
9      newcols[i, 1] = Q[i, j] * Hrefl[1,1] + Q[i, j+1] * Hrefl[2,1]
10     newcols[i, 2] = Q[i, j] * Hrefl[1,2] + Q[i, j+1] * Hrefl[2,2]
11 end
12
13 Q[1:j+1, j:j+1] = newcols

```

² Too many
Qs: TODO re-
name Lanczos
basis vector to
 b_i ?

³ TODO write
pseudocode in-
stead

2.5 Solving the minimum problem, given QR

Given the QR factorization of $\underline{H}_n$, we can transform our minimum problem:

$$\boxed{\min_{z \in \mathbb{R}^{n+1}} \|\underline{H}_n z - e_1\| \|y\|} \quad \text{into} \quad \boxed{\min_{z \in \mathbb{R}^{n+1}} \|R_0 z - Q_0^T(e_1\|y\|)\|}.$$

Proof. Let $\underline{H}_n = QR$, $Q = [Q_0, Q_c]$, $R = [R_0, 0]^T$.

$$\begin{aligned} \|\underline{H}_n z - e_1\| \|y\| &= \|Q^T(\underline{H}_n z - e_1\|y\|)\| && (Q^T \text{ is norm-preserving}) \\ &= \|Q^T QRz - Q^T(e_1\|y\|)\| && (\text{QR factorization of } \underline{H}_n) \\ &= \left\| \begin{bmatrix} R_0 \\ 0 \end{bmatrix} z - \begin{bmatrix} Q_0^T \\ Q_c^T \end{bmatrix} (e_1\|y\|) \right\| && (\text{assumption}) \\ &= \left\| \begin{bmatrix} R_0 z - Q_0^T(e_1\|y\|) \\ -Q_c^T(e_1\|y\|) \end{bmatrix} \right\| && (\text{vector algebra}) \end{aligned}$$

Whose minimum is only influenced by the top portion (the bottom one does not depend on z). $\square$

Once we have transformed the problem to this form, if R_0 is invertible (which is true iff the $\underline{H}_n$ is full-rank), we have obtained a *triangular system*, which is efficiently solved by backsubstitution.

2.6 Computational Complexity

2.6.1 Time Complexity of Lanczos+QR

The algorithm consists of a for-loop of length n (the number of Lanczos iterations). We go through the dominating parts of the j -th step of the loop:

- The Lanczos iteration's complexity, both in its base and inductive cases, is dominated by the matrix-vector multiplication $w := Aq$, which has time complexity $O(m^2)$ (m is the size of A).
- The base case of the QR construction consists of a 2×2 QR factorization, which can easily be considered as a constant-time operation.
- The inductive case of the QR construction is dominated by the $O(j)$ cost of computing the new columns of the Q and R matrices.

This yields a time complexity of

$$O\left(nm + \frac{n(n+1)}{2}\right) = O(nm + n^2)$$

Taking in account the fact that we use a sparse matrix data type in the implementation, we can express the time complexity as follows:

$$O(\text{nnz}(A) + n^2)$$

2.6.2 Space Complexity of Lanczos+QR

Let $\#D$ represent the length of D 's diagonal. We need to store:

- The matrix (in sparse form): $O(\#D + 2 \cdot \text{NNZ}(E))$
- The Lanczos basis: $O(m \cdot n)$
- Q and R : $O(n^2)$

Hence the space complexity is the sum of the above complexities.

2.6.3 Time complexity of GMRES

Our implementation of GMRES consists of:

- A Lanczos+QR computation $O(\text{nnz}(\mathbf{A}) + n^2)$
- Solving an $n \times n$ triangular system: $O(n^2)$

Hence the complexity is dominated by the Lanczos+QR computation $O(\text{nnz}(\mathbf{A}) + n^2)$.

2.6.4 Space complexity of GMRES

Once again, the space complexity is dominated by that of Lanczos+QR, it is therefore:

$$O(\#D + 2 \cdot \text{NNZ}(\mathbf{E}) + mn + n^2)$$

2.7 Stability

The three components of our method are the Lanczos method, the QR factorization and the back substitution used to solve the triangular linear system resulting from the QR factorization.

- **Lanczos** As remarked in [6], “no straightforward theorem is known to the effect that the Lanczos iteration is stable in the sense defined in this book”.
- **QR** As for the QR factorization, we provide a proof of its backward stability.
- **Back substitution** A proof of the backward stability of back substitution can be found in [3].

2.7.1 Background: Backward Stability

An algorithm is said to be *backward stable* if the computed output $\tilde{y}$ can be written as $\tilde{y} = f(\hat{x})$, where:

$$\frac{\|\hat{x} - x\|}{\|x\|} = O(u)$$

where $\hat{x}$ is a perturbed input $x + \Delta x$.

2.7.2 Backward Stability of QR factorization

First, we recall the following result:

Theorem 1 (Backward stability of orthogonal transformations). *If $Q \in \mathbb{R}^{m \times m}$ is orthogonal, and $B \in \mathbb{R}^{m \times n}$, then the computed version $\tilde{C}$ of $C = QB$ is backward stable:*

$$\tilde{C} = Q\hat{B}, \quad \|\hat{B} - B\| \leq O(u)\|B\|$$

The proof is omitted. We can now prove the following:

Claim 1. *Let $A + \Delta_A$ be a perturbed version of the input matrix A . We compute:*

$$\tilde{Q}\tilde{R} = QR(A + \Delta_A).$$

The computed $\tilde{Q}$ and $\tilde{R}$ satisfy:

$$\tilde{Q}\tilde{R} = A + \Delta \quad \text{with} \quad \|\Delta\| \leq O(u)\|A\|.$$

Proof. Recall that:

$$A_0 = A, \quad A_k = Q_k \dots Q_1 A, \quad k \in \{1..n\}$$

Where the Q_k s are orthogonal matrices of the form:

$$\begin{bmatrix} I & 0 \\ 0 & H_k \end{bmatrix}$$

And H_k is a Householder reflector matrix. We first show by induction that:

$$\tilde{A}_k := \tilde{Q}_k(\tilde{A}_{k-1} + \Delta)$$

is such that $\|\Delta\| \leq O(u)\|A\|$ and $\|\tilde{A}_k\| \leq \|A\| + O(u)\|A\|$.

• **Base case** By Theorem 1:

$$\tilde{A}_1 = \tilde{Q}_1(A + \Delta_1), \quad \|\Delta_1\| \leq O(u)\|A\|$$

Additionally:

$$\begin{aligned} \|\tilde{A}_1\| &= \|\tilde{Q}_1(A + \Delta_1)\| \\ &= \|A + \Delta_1\| && \text{(orthogonality)} \\ &\leq \|A\| + O(u)\|A\| && \text{(prop. of norms, above ineq.)} \end{aligned}$$

• **Inductive Case** Let us assume that $\tilde{A}_i = \tilde{Q}_i(\tilde{A}_{i-1} + \Delta_i)$, with:

$$\text{(Hp 1)} \quad \|\Delta_i\| \leq O(u)\|A\| \quad \text{(Hp 2)} \quad \|\tilde{A}_i\| \leq \|A\| + O(u)\|A\|.$$

Then:

$$\tilde{A}_{i+1} = \tilde{Q}_{i+1}(\tilde{A}_i + \Delta_{i+1})$$

We thus have:

$$\begin{aligned} \|\Delta_{i+1}\| &\leq O(u)\|(\tilde{A}_i + \Delta_i)\| && \text{(by Theorem 1)} \\ &\leq O(u)(\|\tilde{A}_i\| + \|\Delta_i\|) && \text{(prop. of norms)} \\ &= O(u)(\|\tilde{A}_i\| + O(u)\|A\|) && \text{(Inductive Hp 1)} \\ &\leq O(u)(\|A\| + O(u)\|A\| + O(u)\|A\|) && \text{(Inductive Hp 2)} \\ &= \|A\|O(u) \end{aligned}$$

And finally:

$$\begin{aligned} \|\tilde{A}_{i+1}\| &= \|\tilde{Q}_{i+1}(\tilde{A}_i + \Delta_{i+1})\| \\ &= \|\tilde{A}_i + \Delta_{i+1}\| && \text{(Orthogonality)} \\ &\leq \|\tilde{A}_i\| + \|\Delta_{i+1}\| && \text{(prop. of norms)} \\ &\leq \|A\| + O(u)\|A\| + \|\Delta_{i+1}\| && \text{(Inductive Hp 2)} \\ &\leq \|A\| + O(u)\|A\| + \|A\|O(u) && \text{(above inequality)} \\ &= \|A\| + O(u)\|A\| \end{aligned}$$

Now recall that $\tilde{R} = \tilde{A}_k$ for some k , and therefore, given:

$$\tilde{Q}\tilde{R} = A + \Delta$$

we can conclude:

$$\begin{aligned} \|\Delta\| &\leq O(u)\|A_k\| && \text{(by Theorem 1)} \\ &\leq O(u)(\|A\| + O(u)\|A\|) && \text{(as shown by induction)} \\ &= O(u)\|A\| \end{aligned}$$

□

3 GMRES with preconditioner

In this section we discuss how we implemented the algorithm using the following preconditioner:

$$P = \begin{bmatrix} D & 0 \\ E & S \end{bmatrix}$$

where $S = ED^{-1}E^T$ or an approximation of it, obtained by replacing the smallest off-diagonal entries of S with zeros. Since our matrix is symmetric, the preconditioned problem we want to solve can be either:

$$PAP^T(P^{-T}x) = Py$$

or

$$P^TAP(P^{-1}x) = P^Ty$$

In our implementation, we provide algorithms to solve both of these systems. As we will later discuss, the second system turned out to provide much more accurate results, but was generally less sparse, hence impacting the compute time.

3.1 Implementation details

Block computation In order to save space, we never construct P explicitly. What we do is exploit the block structure to obtain PAP^T and P^TAP directly:

$$PAP^T = \begin{bmatrix} D^3 & D^2E^T + DE^TS^T \\ (ED + SE)D & (ED + SE)E^T + EE^TS^T \end{bmatrix}$$

$$P^TAP = \begin{bmatrix} (D^2 + E^TE)D + DE^TE & DE^TS \\ E^TED & 0 \end{bmatrix}$$

By precomputing the blocks, we obtain a relatively efficient algorithm.

Approximation of S Once S is computed, we can remove all off-diagonal elements under some threshold using the following Julia code:

```

1      # set off-diagonal elements under some threshold to zero
2      if THRESHOLD_ZERO > 0
3          threshold = THRESHOLD_ZERO
4          mask = .!I(heightE) .& (abs.(S) .< threshold)
5          S[mask] .= 0
6      end

```

3.1.1 Complexity and stability

The results for the computational complexity and stability of the algorithm remain unchanged from those stated previously, for the algorithm without preconditioning. The one difference that had an impact on performance was the increased density of the preconditioned matrices, which makes it so that the preconditioned version of the algorithm takes longer to compute.

4 Additional implementations

4.1 Restarted versions of the algorithms

In addition to the GMRES implementation, we provided a *higher-order function* that takes as input any GMRES implementation, and uses it to perform the **restarted GMRES** algorithm, proposed as an exercise in chapter 35 of [6] and further discussed in [5].

Reasons for using the restarted version For larger values of n , the cost of GMRES in operations and storage may be prohibitive. This is because, as seen before, the algorithm is quadratic both in time and in space. One way to mitigate these issues is to *restart* the algorithm after a certain number of iterations. The restarted GMRES algorithm, as implemented in Julia, is as follows:

```
1 function RestartedGMRES(GMRES_Method, A, y, max_iter, restart, tol)
2
3     x = zeros(size(A)[2])
4
5     # Initial guess is initial r0
6     r = y - A * x
7     norm_r = norm(r)
8
9     iter = 0
10
11     while iter < max_iter && norm_r > tol
12
13         # Perform GMRES and get current solution
14         dx = GMRES_Method(A, y, restart)
15
16         x += dx
17         r = y - A * x
18         norm_r = norm(r)
19
20         iter += restart
21     end
22
23     if norm_r > tol
24         println("Did not converge")
25     end
26
27     return x
28 end
```

Correctness: restart invariant Let r_i be the residual after restart i : $r_i = y - Ax_i$. We have the following:

$$Ax_i = y - r_i$$
$$x' := \text{GMRES}(A, r_i) \implies Ax' = \tilde{r}_i \approx r_i$$

Let $x_{i+1} = x_i + x'$, now:

$$A(x_{i+1}) = Ax_i + Ax' = y - \tilde{r}_i + r_i \approx y$$

This shows that, at each restart:

$$Ax_i \approx y.$$

Indeed, we should ensure that, under some assumptions, the approximations grow in precision with each iteration, but we do not go into this: with the right parameter choice, we found experimentally that the method provides very good approximations of the solution in very brief time.

Drawbacks of the method A well known difficulty with the restarted GMRES algorithm is that it can stagnate when the matrix is not positive definite [5]. Indeed, experimentally, we have noticed that choosing the wrong parameters (**restart** and tolerance) would yield stagnation. However, as evident from the experimental data, the restarted algorithm often delivered solutions orders of magnitude more fast and precise than the bare implementations.

5 Generating instances of the problem

5.1 Min-Cost Flow

The problem arises from the KKT system of the convex quadratic separable Min-Cost Flow Problem. The problem is defined as follows:

- Let $G = (V, A)$ be a graph, with V its set of nodes, and A its set of arcs.
- Let $b \in \mathbb{R}^{|V|}$ be a vector of *node flows* (based on which we can classify nodes as source, target or transshipment).
- Let $x \in \mathbb{R}^{|A|}$ be a *flow assignment*:

$$x = \begin{bmatrix} \varphi_1 \\ \vdots \\ \varphi_{|A|} \end{bmatrix}$$

- Let φ_{ij} be the flow on the edge $(i, j) \in A$.
- Let $C_{ij} : \mathbb{R} \rightarrow \mathbb{R}$ be a quadratic function that maps a flow φ to the *cost* of transporting such flow through the (i, j) edge.

We formulate the problem as follows:

$$\min \sum_{(i,j) \in E} C_{ij}(\varphi_{ij})$$

With the constraint given by Kirchhoff's law of conservation of flow:

$$\forall i \in V. \sum_{\substack{i \in V \\ (j, i) \in A}} \varphi_{i,j} - \sum_{\substack{i \in V \\ (i, j) \in A}} \varphi_{i,j} = b_i.$$

There would be other constraints, e.g. the non-negativity of flows or per-edge upper and lower bounds to the flows, but in order to obtain a KKT system of the requested shape we do not consider these.

5.2 Problem and constraints in matrix form

We mentioned that each function $C_{ij}(\varphi)$ is quadratic. Let us write it as:

$$\forall e \in A. C_e(\varphi) = \frac{1}{2}c_e\varphi^2 + d_e\varphi$$

We can thus rewrite the minimum problem as:

$$\min (x^T D x + \langle d, x \rangle)$$

Where:

$$D = \begin{bmatrix} c_1 & & 0 \\ & \ddots & \\ 0 & & c_{|A|} \end{bmatrix} \quad d = \begin{bmatrix} d_1 \\ \vdots \\ d_{|A|} \end{bmatrix}$$

Additionally, given the incidence matrix $\underline{E}$, we can write the constraint as simply $\underline{E}x = b$.

5.3 KKT System

From the above problem and equality constraint, we obtain the following system:

$$\underline{E}x = b \tag{KKT-F}$$

$$Dx + \underline{E}^T \cdot \mu = -d \tag{KKT-G}$$

With $\mu \in \mathbb{R}$. The system can be rewritten as:

$$\begin{bmatrix} D & \underline{E}^T \\ \underline{E} & 0 \end{bmatrix} \begin{bmatrix} x \\ \mu \end{bmatrix} = \begin{bmatrix} -d \\ b \end{bmatrix}$$

In order to obtain an equivalent full-rank system, as suggested in the problem statement, we can remove one row from the incidence matrix.

5.4 Netgen dataset, and generating D

In order to obtain realistic instances to test our algorithms, we used the netgen [4] dataset. This provides graphs in the DIMACS format defined in [1], coming with the following information:

- Node flow of each vertex
- Source, target, capacity, and *linear* cost of each edge.

The main issue with this dataset is that, indeed, it only provides linear costs.

Generating D Frangioni et al. [2] propose a procedure for generating realistic quadratic costs. Their approach is to generate the costs uniformly in $[ac_{ij}, bc_{ij}]$, where c_{ij} are the linear costs and a, b are integer numbers. The interval considered in the paper was $[3c_{ij}, 10c_{ij}]$.

Other ways to construct D In order to further explore the performance of our algorithm, we will additionally test with other versions of the matrix D , namely:

- Diagonal with all ones
- Diagonal with entries uniformly distributed in $[0, 1]$

6 Experimental Results

The following experimental results refer to a 9000×9000 matrix, and were ran on a laptop with a *13th Gen Intel(R) Core(TM) i5-1335U* processor, and 16 GB of RAM.

6.1 Comparison in performance for different D

As outlined in Figure 1, we obtained the following results:

- With $D = I$, the algorithm reaches convergence to 10^{-10} in fewer than 200 steps
- With the uniform distribution, the algorithm settles on 10^{-8} in almost 1500 steps.
- The matrix with the generated quadratic costs is quite ill-conditioned, with a condition number of $1.347 \cdot 10^{11}$. Therefore, the algorithm does not quite converge at all.

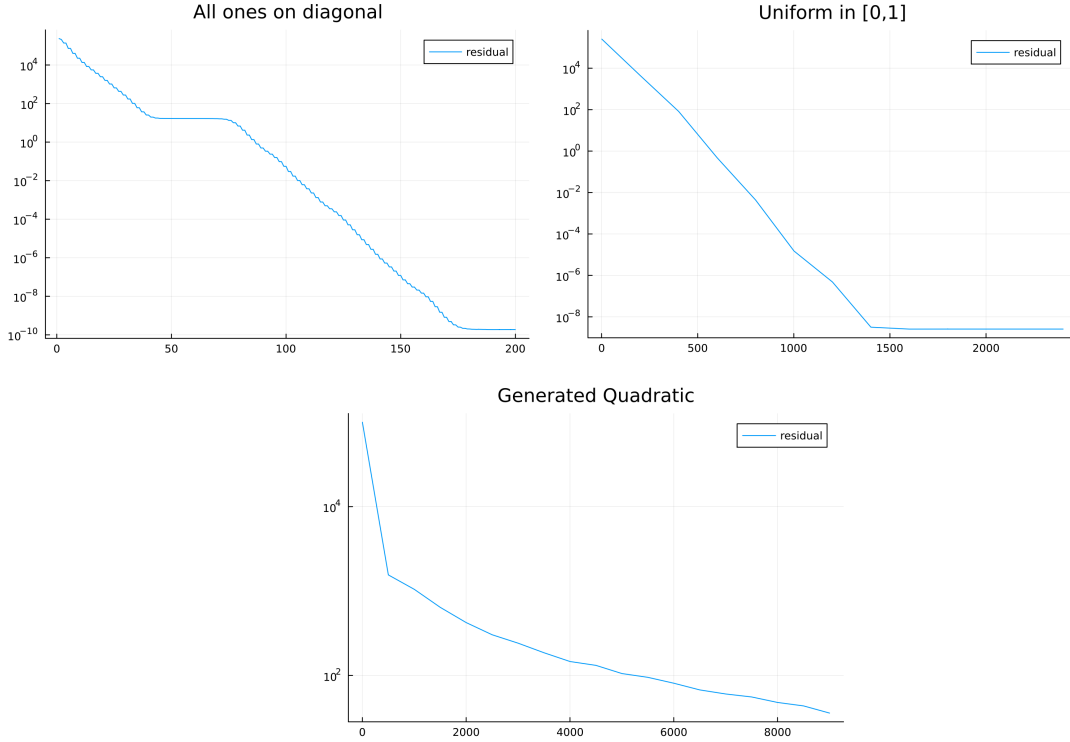


Figure 1: Residuals vs iterations for different constructions of D .

6.2 Comparison between the preconditioned and non-preconditioned systems

We proposed two ways to use the preconditioner, preserving the symmetry:

$$PAP^T(P^{-T}x) = Py$$

and

$$P^TAP(P^{-1}x) = P^Ty$$

We analyse the impact of both. In Figures 2 and 3, we show how:

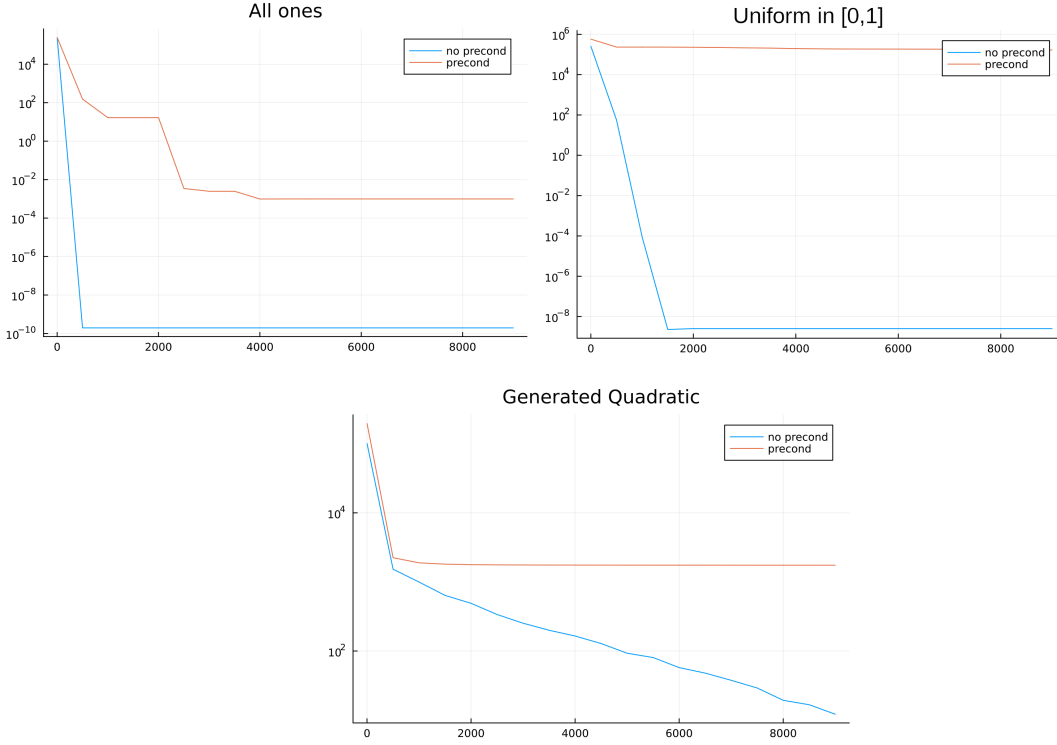


Figure 2: Comparison between preconditioned and non-preconditioned system in PAP^T form.

- For the matrix with $D = I$, the preconditioned system in P^TAP form is solved with slightly better precision than the PAP^T one. Nonetheless, both preconditioned methods achieve much worse precision than the non-preconditioned ones.
- For the matrix with uniformly distributed diagonal elements, the preconditioned method fails to converge in both cases.
- Similarly, with the generated quadratic costs, the residual stagnates on a very high value, without reaching an acceptable value.

From these experimental results, it is apparent that the preconditioned system converges to a less accurate result than the non-preconditioned one. This is not what we expected, but it would seem that the proposed preconditioner is not adequate for this application.

In order to investigate this behaviour, we compute the matrix conditioning for the preconditioned and non-preconditioned matrices:

	Non-precond.	PAP^T	P^TAP
All ones	$7.104 \cdot 10^3$	$3.432 \cdot 10^{12}$	$3.303 \cdot 10^{11}$
Uniform in $[0, 1]$	$2.74 \cdot 10^3$	$1.918 \cdot 10^{14}$	$1.182 \cdot 10^{13}$
Generated Quadratic	$1.347 \cdot 10^{11}$	$6.496 \cdot 10^{25}$	$1.041 \cdot 10^{29}$

Indeed, we observe that the preconditioned matrices exhibit higher condition numbers than the non-preconditioned ones. This is consistent with the preconditioner not being adequate.

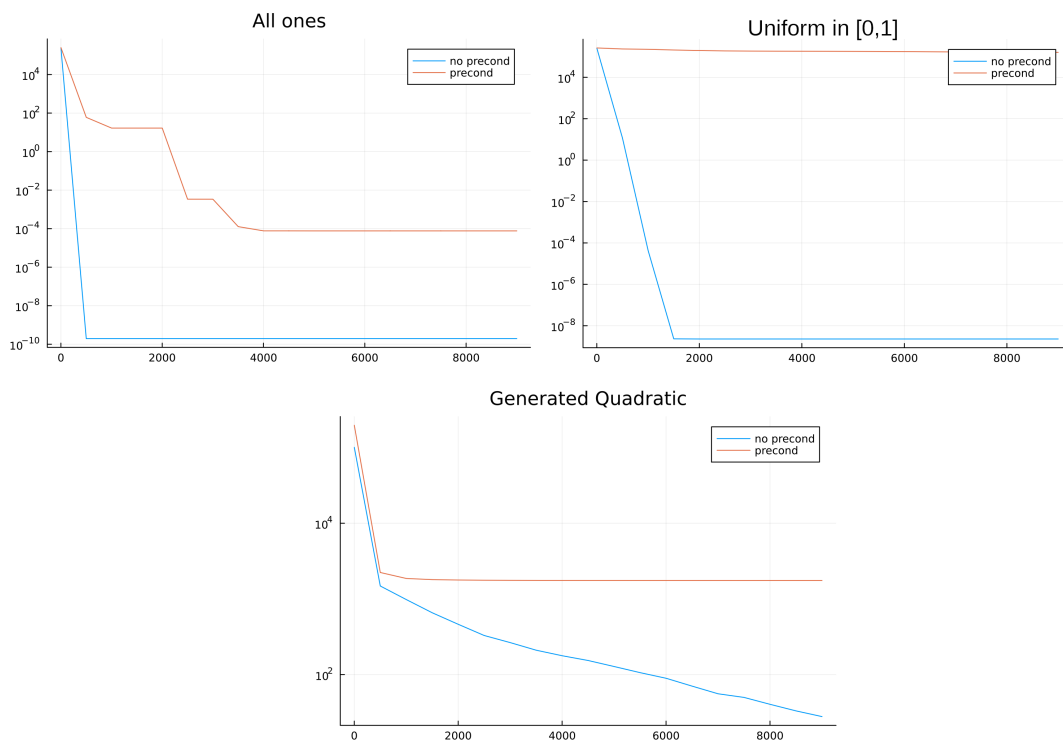


Figure 3: Comparison between preconditioned and non-preconditioned system in $P^T A P$ form.

6.3 Comparison between GMRES and Restarted GMRES

[TODO]

References

- [1] *DIMACS minimum cost flow problems*. URL: https://web.mit.edu/lpsolve_v5525/doc/DIMACS_mcf.htm.
- [2] Antonio Frangioni et al. “Projected perspective reformulations with applications in design problems”. In: *Operations research* 59.5 (2011), pp. 1225–1232.
- [3] Nicholas J Higham. *Accuracy and stability of numerical algorithms*. SIAM, 2002.
- [4] *NETGEN*. URL: <https://ngsolve.org/>.
- [5] Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- [6] Lloyd N Trefethen and David Bau. *Numerical linear algebra*. SIAM, 2022.